HQDD1 通信仕様書(CAN / UART)

対象: hqdd_1 基板(STM32G431KB + DRV8311H + AS5600L + TCAN330)のファームウェア `firmware/` 作成日: 2026-10-17

この文書は、基板が外部から受け付けるコマンドと、基板が出力するデータの仕様をまとめたものです。内容はファームウェアのソース(`firmware/src/main.cpp`、`firmware/src/can_link.cpp`、`firmware/src/settings_store.cpp`)に基づいています。

---

1. 概要

項目	CAN	UART
物理	FDCAN1 / TX=PA12, RX=PA11 / トランシーバ TCAN330	USART2 / TX=PA2, RX=PA15 / 3.3V TTL(コネクタ J3)
速度	1 Mbps(クラシック CAN 互換、CAN FD は使わない)	115200 bps、8N1
宛先の指定	CAN ID(`0x100 + ID`)	行頭の `@<ID>`(省略可)
コマンド	1 フレーム = 1 コマンド(ASCII、最大 8 文字)	1 行 = 1 コマンド(LF 終端、最大 47 文字)
テレメトリ	20 ms 周期の 8 バイトバイナリ(自動送信)	200 ms 周期のテキスト 1 行(自動送信、停止可)
値の読み出し	問い合わせコマンド `?XX` に CAN フレームで応答	問い合わせコマンド `?XX` に 1 行で応答

1.1 「コマンド」とは

この基板への指示は、**「コマンド」と呼ぶ短い文字列**で行います。例: `M2`(目標を 2 にする)、`S`(停止)、`W`(設定を保存)、`F`(設定を出荷状態に戻す)。コマンドの先頭 1〜2 文字が命令の種類を表します。この文書の `` `W` `` や `` `F` `` のような表記は、**キーボードのキーではなく、基板に送る文字列そのもの**を指します。

コマンドの送り方は 3 通りです。**中身は同じ文字列**で、運び方が違うだけです。

送り方	方法	`W`(保存)を送る例
UART(シリアル端末)	端末ソフト(PlatformIO の `pio device monitor` など)で 1 行入力して Enter。行頭に宛先 `@<ID>` を付ける	`@1 W` + Enter
CAN	CAN フレームのデータ部に、文字列を ASCII の 16 進数で入れる。ID は `0x100 + ID`	`cansend can0 101#57`(`57` は文字 `W` の ASCII コード)
Web UI	ボタンやスライダーを操作する。**UI が、対応するコマンドを自動で送る**	「フラッシュに保存」ボタン

Web UI を使う場合は、コマンドを自分で打つ必要はありません。3 章の表に、各コマンドに対応する UI の操作を載せています。UI の下部にある「生コマンド」欄には、コマンドを直接入力することもできます。

1.2 用語

用語	意味
デバイス ID(ID)	基板ごとの番号(0〜7)。複数台を区別するために使う。CAN ID(通信フレームの識別子)とは別のもので、CAN ID は `0x100 + デバイス ID` のように、これから作られる
テレメトリ	基板が自動で送ってくる状態データ(角度・速度・電流など)

用語	意味
出力軸 / モーター軸	出力軸は減速機の出力側(アクチュエータの先端)、モーター軸はモーター本体の回転軸。減速比 `GR` の分だけ値が違う
フラッシュ	基板のマイコン内の不揮発メモリ。ここに保存した値は、電源を切っても残る
出荷状態 / 出荷値	基板に書き込んだプログラム(コード)に書かれている、パラメータの初期値
LiPo、2S、3S	リチウムポリマー電池。S は直列のセル数で、**2S = 2 セル直列(公称 7.4 V、満充電 8.4 V)**、**3S = 3 セル直列(公称 11.1 V、満充電 12.6 V)**

どちらのインターフェース(CAN / UART)も、同じコマンド文字列を使います。違いは、コマンドの運び方と、応答・テレメトリの形式です。

デバイス ID

- ID は 0〜7 です。基板上のボタン(PA3)で設定します。2 秒長押しで ID 設定モードに入り、押した回数(1〜8)で ID(= 回数 − 1)を決めてフラッシュに保存します。
- LED1(PA5)が、現在の ID を (ID + 1) 回の点滅で表示します。
- ID を変更すると、CAN の受信フィルタはその場で更新されます。
- **プログラムを書き込み直すと、ID はコード上の既定値(`firmware/src/device_id.h` の `DEFAULT_DEVICE_ID` 、現在 1)に戻ります。** ボタンで設定した ID は、書き込みのたびに失われます。複数台を書き込むときは、台ごとに既定値を変えてビルドするか、書き込み後にボタンで設定し直してください。

---

2. 共通事項

2.1 単位と座標

量	基準軸	単位	備考
コマンドの目標値 (`M<値>`)	出力軸	位置 rad / 速度 rad/s / トルク モードは下記	減速比で換算してモーターへ渡す
テレメトリの角度・速度	出力軸	rad、rad/s	角度は `Z` で再定義した 0 位置が基準
Iq、Id、Vq	モーター軸 (生値)	A、A、V	減速比換算なし
トルク	出力軸	Nm	`Iq × Kt ×` 減速比 (Kt = 0.0367 Nm/A、理論値で ±10〜20% の誤差を見込む)

トルクモードの目標値は、「出力軸換算電流」です。`M<値>` の値を減速比で割ったものがモーターの電流指令[A]になります。出力トルクの目安は `値 × 0.0367` Nm です。

例: 減速比 29.16 のとき、`M10` はモーター電流 0.343 A(出力トルク約 0.37 Nm)です。

2.2 制御モード

`MC<n>` で切り替えます。

n	モード	目標値の単位
0	トルク(電流)	出力軸換算電流(上記)
1	速度	rad/s(出力軸)
2	位置	rad(出力軸)

n = 3〜5(開ループ等)は SimpleFOC の標準機能で、本仕様の対象外です。

モード切替時の目標値の扱い。

- 位置モードに入った時は、目標が現在位置になります。その場で保持し、0 位置へ飛びません。
- 速度・トルクモードに入った時は、目標が 0 になります。
- どの場合も、切り替えと同時に制御器の積分値などをリセットします。

2.3 パラメータ表

名前は 2 文字です。読み出し(`?XX`)と設定(`PXX<値>`)に共通で使います。範囲外の値は無視されます。

名前	内容	単位	範囲	出荷値	備考
`VP`	速度 P ゲイン		0〜10000	0.03	`MVP` でも設定可
`VI`	速度 I ゲイン		0〜10000	0.03	`MVI`
`VD`	速度 D ゲイン		0〜10000	0	`MVD`
`VF`	速度 LPF 時定数	s	0〜1	0.008	`MVF`
`AP`	位置 P ゲイン		0〜10000	30	`MAP`
`QP`	電流 P ゲイン(q・d 共通)	V/A	0〜10000	0.8	`MQP` 。d 軸は常に q と同じ値
`QI`	電流 I ゲイン(q・d 共通)		0〜100000	60	`MQI`
`GR`	減速比		0.1〜1000	29.16	目標値・テレメトリの換算に使う
`IL`	電流上限	A	0.05〜5	1.0	トルク指令と速度 PID の出力を制限
`VL`	速度上限	rad/s(**モーター軸**)	1〜300	60	出力軸では `値 ÷ GR`
`UL`	電圧上限	V	0.5〜17	6.0	電流 PID の出力上限。実際は電源電圧以下に制限される
`SV`	電源電圧	V	5〜17	8.0	LiPo バッテリーに合わせる。2 セル(2S)は 8、3 セル(3S)は 12
`AV`	起動時の整列電圧	V	0.5〜12	5.0	**次回起動から有効**

- **電源電圧 `SV`** : 基板に電源電圧を測る回路はないので、手動で設定します。実際の電圧と違うと、電流ループのゲインがずれます。LiPo 3 セル(3S)で `SV` を 8 のままにすると、実効ゲインが約 1.4〜1.6 倍になり、発振する恐れがあります。
- **出荷状態**は、表の「出荷値」の列、つまり基板に書き込むコードの初期値です。

2.4 設定の保存と出荷状態への復帰

パラメータ(2.3 の 13 項目)の変更は、**送った瞬間に反映**されます。ただし、基板のメモリ(RAM)に置かれただけなので、**そのままでは電源を切ると消えます**。電源を切っても残したい場合は、フラッシュに保存します。この操作のためのコマンドが、`W` と `F` です。

やりたいこと	コマンド(基板に送る文字列)	Web UI での操作	UART での入力例(ID = 1)	CAN での送信例(ID = 1)
今の値をフラッシュに保存する	`W`	「設定の保存・初期化」の **「フラッシュに保存」**	`@1 W`	`cansend can0 101#57`
全パラメータを出荷状態に戻す	`F`	同じ欄の **「出荷状態に戻す」**	`@1 F`	`cansend can0 101#46`

- `W` **(Write = 保存)**: パラメータ表の 13 項目すべてを、フラッシュ(ページ 62)に保存します。次回の起動時に読み込んで適用します。
- `F` **(Factory = 出荷状態)**: 全パラメータを出荷値(コード上の初期値)に戻し、フラッシュの保存内容も消します。
- `W` と `F` は、**モーター軸の速度が 2 rad/s を超えているときは実行されません**(拒否の応答が返ります)。フラッシュの書き込み中は、数十 ms の間、制御が止まるためです。モーターが止まっている状態で行ってください。
- 応答は、UART では `SAVED` や `RESET OK` などの 1 行、CAN では `0x300 + ID` のフレーム(4.5)です。Web UI では、結果がログ欄に表示されます。

プログラムを書き込み直したとき。 - 保存内容は無効になり、**コード上の値(出荷値)が使われます。** ビルドごとに固有の番号を保存データに付けていて、番号が違うデータは読み込まれません。 - コードで初期値を変えたら、そのまま書き込めば反映されます。 - 調整済みの値を書き込み後も使いたい場合は、その値をコードの初期値に反映してから書き込んでください(UI の「全て読込」で、現在値が確認できます)。 - 電源の入れ直しやリセットでは、保存内容は保持されます(同じプログラムのままの場合)。

位置の 0 点(`Z`)は、保存されません。

---

3. コマンド一覧(CAN・UART 共通の書式)

`<…>` は値、`[…]` は省略可を表します。「Web UI での操作」の列は、そのコマンドを Web UI のどの操作が送っているかを示します。

コマンド	内容	応答	Web UI での操作
`M<値>`	目標値を設定(モードの単位。2.1、2.2 参照)	UART に `Target: <値>` (行頭 ID なし)。CAN は応答なし	スライダー、「送信」ボタン、「0」ボタン
`M`	現在の目標値を返す	UART に値のみ。CAN は応答なし	(なし)
`MC<n>`	制御モードを切り替え(0 / 1 / 2)	応答なし	「トルク」「速度」「位置」ボタン
`ME1` / `ME0`	モーターの有効 / 無効	応答なし	「Enable」「Disable」ボタン
`MVP<値>` `MVI` `MVD` `MVF` `MAP` `MQP` `MQI`	ゲインを設定(SimpleFOC 標準コマンド)	UART に SimpleFOC 標準の確認行(設定後の値を表示)	「PIDゲイン」欄の「設定」ボタン

コマンド	内容	応答	Web UI での操作
`S`	停止。速度モードで目標 0 にして減速する	応答なし	「停止」ボタン、Esc キー
`Z`	現在位置を出力軸の 0 位置にする	応答なし	位置モードの「現在位置を0にリセット」ボタン
`?<名前>`	パラメータの現在値を読む(例 `?VP`、`?GR`)。制御モードは `?MC`	値の応答(CAN: 0x300+ID のフレーム / UART: `GAIN` 行)。不明な名前は CAN で結果 = 0、UART で `GAIN ?`	各項目の「読」ボタン、「全て読込」ボタン、接続直後の自動読み出し
`P<名前><値>`	パラメータ表の項目を設定(例 `PIL1.5`、`PSV12`、`PGR29.16`)	UART に `GAIN` 行(設定後の値)。CAN は応答なし。範囲外は無視	「制限・システム設定」欄の「設定」ボタン、電源電圧プリセット
`G<減速比>`	`PGR<値>` の別名。`G` のみで現在値を返す	`P` と同じ	(使わない)
`W`	パラメータをフラッシュに保存(2.4)	CAN: 0x300+ID の `WS` フレーム / UART: `SAVED` 等の行	「フラッシュに保存」ボタン
`F`	全パラメータを出荷状態に戻す(2.4)	CAN: 0x300+ID の `FR` フレーム / UART: `RESET OK` 等の行	「出荷状態に戻す」ボタン
`T1` / `T0`	UART の周期テレメトリを ON / OFF(CAN には影響しない)	応答なし	UART 接続時に自動で送る(5.5)
`L`	高速キャプチャ(1 kHz × 600 点)を開始。結果は **UART にのみ**出力	UART に CSV(6 章)	「高速キャプチャ(L)」ボタン(UART 接続時のみ)

注意。

- 表に載せたもの以外の SimpleFOC 標準コマンド(`MLC`、`MR`、`MK` など)も使えますが、値の換算や範囲確認は行われません。また、`MLC` などは、パラメータ表の値(`IL` など)を更新しないので、`P` による設定と食い違います。制限値は `P` コマンドで設定してください。
- `?` は SimpleFOC では「コマンド一覧の表示」に使われますが、本ファームでは、先頭が `?` の行を問い合わせとして先に処理します。`@`(UART の宛先指定)も同様に、先に宛先として解釈します。

---

4. CAN 仕様

4.1 物理・タイミング

- クラシック CAN 互換、1 Mbps、標準 11 ビット ID。
- 終端抵抗 120 Ω をバスの両端に入れてください。
- 受信は標準フレームで、ID が `0x100 + ID` に一致するものだけを受け取ります。標準フレームのそれ以外の ID は破棄します。
- 受信の処理は、メインループの 1 周ごとに 1 フレームです。ループは実機で約 2 kHz(1 周 ≈ 0.5 ms)でした。連続して送る場合は、1 ms 以上の間隔を空けてください。付属の Web UI は 60 ms 間隔で送り、全パラメータの読み出しが問題なく動くことを確認しています。

4.2 ID の割り当て

方向	CAN ID	内容
PC → 基板	`0x100 + ID`	コマンド(4.3)
基板 → PC	`0x200 + ID`	テレメトリ(4.4)。20 ms 周期で自動送信
基板 → PC	`0x300 + ID`	コマンドへの応答(4.5)。`?`、`W`、`F` のときだけ送信

ID は 0〜7 なので、最大 8 台を同じバスに接続できます。

4.3 コマンドフレーム(`0x100 + ID`)

- DLC = 文字数(1〜8)。データは ASCII 文字列で、終端文字は不要です。
- 9 文字以上のコマンドは送れません。ゲインなどの値は、8 文字に収まる桁数にしてください(例: `MVP0.015` は 8 文字で収まります)。
- 本体は 3 章のコマンドをそのまま実行します。UART 用の `@<ID>` は CAN では使いません。

例(`ID = 1`、Linux の `can-utils`):

```
cansend can0 101#4D4331       # "MC1": 速度モードへ
cansend can0 101#4D33         # "M3" : 目標 3
cansend can0 101#53           # "S"  : 停止
cansend can0 101#3F5650       # "?VP": VP を読む -> 0x301 に応答
```

4.4 テレメトリフレーム(`0x200 + ID`)

DLC = 8。リトルエンディアンの `int16` が 4 つ。

バイト	内容	スケール	単位	範囲
0–1	角度(出力軸)	実値 × 1000	rad	±32.767
2–3	速度(出力軸)	実値 × 100	rad/s	±327.67
4–5	Iq(モーター軸)	実値 × 1000	A	±32.767
6–7	トルク(出力軸)	実値 × 1000	Nm	±32.767

- 範囲を超えた値は、±32767 に飽和します。
- 目標値、Id、Vq、制御モード、ループ周波数は含まれません。目標値は、PC 側で送った値を覚えておいてください。制御モードは `?MC` で読めます。
- 例: `B6 FF 00 00 01 00 01 00` → 角度 = `0xFFB6` = −74 → −0.074 rad、速度 0、Iq = 0.001 A、トルク = 0.001 Nm。

4.5 応答フレーム(`0x300 + ID`)

DLC = 8。

`?<名前>` への応答

バイト	内容
0–1	名前(ASCII 2 文字。問い合わせの名前をそのまま返す)
2	結果。1 = 成功、0 = 不明な名前

バイト	内容
3	0
4–7	値(リトルエンディアンの `float32`)。不明な名前のときは 0

`?MC` の値は制御モード(0.0 / 1.0 / 2.0)です。

例: `?VP` の応答 `56 50 01 00 8F C2 F5 3C` → `VP` = 0.03。

`W` (保存)への応答: `57 53 <結果> 00 00 00 00 00` (`WS`)。結果 = 1:成功、0:失敗、2:回転中のため拒否。

`F` (出荷状態)への応答: `46 52 <結果> 00 00 00 00 00` (`FR`)。結果の意味は `W` と同じです。

応答のないコマンド: `M`、`MC`、`ME`、`P`、`G`、`S`、`Z`、`T`、`L` と、SimpleFOC 標準のコマンドは、CAN 上に応答を返しません。これらの文字列応答は UART にのみ出ます。`P` で設定した結果を CAN で確かめるには、続けて `?<名前>` を送ってください。

---

5. UART 仕様

5.1 物理・フォーマット

- 3.3V TTL、115200 bps、8N1。USB-TTL アダプタを J3 に接続します。アダプタの TXD → 基板 PA15(回路図ラベル `UART_TX`)、アダプタの RXD → 基板 PA2(同 `UART_RX`)、GND を共通にします。MCU から見て PA2 が送信、PA15 が受信です。
- 受信は ASCII のテキスト行で、LF(`\n`)で確定します。CR は無視されます。最大 47 文字で、超えた行は破棄されます。
- 基板が出力する行も LF(`\r\n`)終端のテキストです。

5.2 宛先の指定

複数台を同じバスに載せるため、コマンド行の先頭に宛先を付けます。

書式	動作
`@<ID> <コマンド>`	その ID の基板だけが実行する(例: `@3 M0.3`)
`@* <コマンド>`	全台が実行する。**使えるのは `S` (停止)と `T` (テレメトリ ON/OFF)だけ**。それ以外は無視される
`<コマンド>` (`@` なし)	接続している基板が、ID に関係なく実行する。端末から 1 台を直接操作するための従来形式。複数台のバスでは使わないこと

`@` の直後に ID を置き、続けて空白を入れてください(`@3 M0.3`)。ID が違う基板は、何も出力しません。

5.3 基板からの出力行

行頭に `#<ID>` が付く行と、付かない行があります。

種類	形式	説明
テレメトリ	`#<ID> v=… tgt=… Iq=… Id=… Vq=… sa=… trq=… loopHz=…`	200 ms 周期。5.4 参照

種類	形式	説明
パラメータ値	`#<ID> GAIN <名前> <値>`	`?`、`P`、`G` の応答。値は小数 5 桁
不明な名前	`#<ID> GAIN ?`	`?` または `P` の名前が不明
保存結果	`#<ID> SAVED` / `SAVE FAILED` / `SAVE BUSY (回転中)`	`W` の応答
出荷状態の結果	`#<ID> RESET OK` / `RESET FAILED` / `RESET BUSY (回転中)`	`F` の応答
その他	(行頭 ID なし)	`Target: <値>`、SimpleFOC の確認行、キャプチャ、起動メッセージ

受信側のプログラムは、`#<ID>` で始まる行を、自分が見ている ID の分だけ使ってください。

5.4 テレメトリ行

```
#1 v=0.00 tgt=0.00 Iq=0.01 Id=0.00 Vq=0.09 sa=2.04 trq=0.0057 loopHz=2000
```

キー	内容	単位	基準
`v`	速度	rad/s	出力軸
`tgt`	現在の目標値	モードの単位(2.1、2.2)	出力軸
`Iq`	q 軸電流	A	モーター軸
`Id`	d 軸電流	A	モーター軸
`Vq`	q 軸電圧	V	モーター軸
`sa`	角度	rad	出力軸
`trq`	トルク	Nm	出力軸(小数 4 桁)
`loopHz`	制御ループの周波数	Hz	

- 約 200 ms 周期で出力されます。`T0` で停止し、`T1` で再開します。基板の起動直後は ON です。
- 高速キャプチャ(`L`)の実行中は、出力されません。

5.5 複数台を同じバスで使う手順

1. 接続した直後に、`@* T0` を送って、全台の周期テレメトリを止めます。
2. 続けて、見たい基板に `@<ID> T1` を送ります。その基板のテレメトリだけが流れます。
3. 別の基板に切り替えるときは、手順 1、2 を繰り返します。
4. 緊急停止は `@* S` です。全台が停止します。

電気的な注意。 本ファームウェアは TX ピン(PA2)を常時プッシュプルで駆動し、方向制御(DE)の機能はありません。複数台の TX を直結すると、信号がぶつかります。1 本のバスで共有する場合は、RS-485 などのバストランシーバ(半二重)を用いて、TX の出力を衝突させない構成にしてください。この点は、ここで挙げた手順(テレメトリを黙らせる)では解決できません。

5.6 起動メッセージ

電源投入から数秒の間、行頭 ID のない行が出力されます。

```
=== hqdd_1 (STM32G431KBUx) CURRENT-mode FOC ===
I2C scan:
  found 0x36
sensor check (手で軸を回すと値が変わるはず):
  ang=...
settings loaded from flash        # 保存済みの設定があるときだけ
ready. CURRENT-torque: ...
```

この間、コマンドは受け付けられません(`ready.` の表示後から有効)。

---

6. 高速キャプチャ(`L`、UART のみ)

PID 調整用に、1 kHz で 600 点(約 0.6 秒)を記録して出力します。CAN から `L` を送って開始することもできますが、結果は UART にのみ出ます。

```
CAPBEGIN
<t_us>,<v>,<Iq>,<Vq>,<sa>
...(600 行)
CAPEND
```

列	内容	単位	基準
`t_us`	最初のサンプルからの経過時間	µs	
`v`	速度	rad/s	**モーター軸**(生値)
`Iq`	q 軸電流	A	モーター軸
`Vq`	q 軸電圧	V	モーター軸
`sa`	角度	rad	**モーター軸**(生値)

- 行頭 ID は付きません。キャプチャ中は、周期テレメトリが止まります。
- 目標を変える操作(例: `M3`)の直前に `L` を送ると、ステップ応答が記録できます。

---

7. 動作上の注意

- **通信が途絶えたときの動作はありません。** 基板には、CAN・UART の通信断を検出して止める機能がありません。最後に受けた指令のまま動き続けます。上位側で、定期的な送信と、停止(`S`、UART は `@* S`)の手順を用意してください。
- **コマンドの実行順序**: CAN は 1 周につき 1 フレーム、UART は受信した行を順に処理します。モード切替(`MC`)と目標値(`M`)を続けて送る場合は、この順序で送ってください。
- **位置モードの目標値**は、`Z` で再定義した 0 位置を基準にします。`Z` を送ると、位置モードの目標も 0 になり、その場で保持されます。
- **減速比 `GR` の変更**は、換算の基準が変わります。位置の意味がずれるので、停止中に行い、変更後に `Z` で 0 位置を取り直してください。
- **保存(`W`)・初期化(`F`)は停止中に行う**こと。回転中は拒否されます(2.4)。

---

8. 通信シーケンスの例

8.1 CAN で速度 2 rad/s を指令し、ゲインを読む(ID = 1)

手順	送信	結果
1	`0x101` ← `ME1`	モーター有効
2	`0x101` ← `MC1`	速度モード(目標は 0)
3	`0x101` ← `M2`	目標 2 rad/s
4	`0x101` ← `?VP`	`0x301` に VP の値が返る
5	`0x101` ← `S`	停止

8.2 UART で ID 3 を操作する

```
@* T0              全台のテレメトリを停止
@3 T1              ID 3 のテレメトリを開始
@3 ME1
@3 MC1
@3 M2              速度 2 rad/s
@3 ?VP             -> #3 GAIN VP 0.03000
@3 PVP0.04         -> #3 GAIN VP 0.04000
@3 W               -> #3 SAVED
@* S               全台停止
```

8.3 LiPo バッテリーを 2 セル(2S)から 3 セル(3S)に替える

基板には電源電圧を測る回路がないので、バッテリーを替えるときは、電源電圧の設定 `SV` を手動で合わせます。

1. 3S を接続する**前に**、2S のまま `PSV12` を送ります(Web UI では電源電圧の「3セル(3S) 12V」ボタン)。実際の電圧より低い値を設定しても、出力が小さめになるだけで安全側です。
2. 動作を確かめたら、`W` で保存します(2.4)。ただし、**プログラムを書き込み直すと出荷値の 8 V に戻る**ので、その場合は設定し直してください。
3. LiPo バッテリーを 3S に替えます。
4. 他のパラメータ(PID ゲインなど)は、変更不要です。

---

9. 関連ファイル

ファイル	内容
`firmware/src/main.cpp`	コマンド処理、パラメータ表、UART の行処理、テレメトリ出力
`firmware/src/can_link.cpp` / `can_link.h`	CAN の送受信、ID の割り当て、応答フレーム
`firmware/src/settings_store.cpp`	パラメータのフラッシュ保存
`firmware/src/device_id.cpp` / `ui.cpp`	ID の保存、ボタンによる ID 設定
`webui/`	この仕様に従う Web UI(`server.py` と `static/index.html`)